IPFS Integration Guide & Chain/Surface Mapping for the PYTHAI Project Suite

TL;DR

- **Self-host a Kubo + IPFS Cluster core under rootless Podman (Quadlet/systemd), pin everything with CIDv1, reference content on-chain via `ipfs://<cid>`, and use IPFS+Filecoin for durability while reserving Arweave/AR.IO for true write-once permanence** — this satisfies the cypherpunk2048 standard (Apache-2.0, no admin keys, no proxies, mainnet-only, Podman-over-Docker) end to end.
- For the three live surfaces: **AgenticPlace** content-addresses ERC-8004 agent registration files / agent cards / marketplace listings (CID in the ERC-721 `tokenURI`/`agentURI`); **mindX** content-addresses every API artifact and model output with a signed CID receipt; **BANKON** content-addresses identity documents and DAIO governance proposals/votes (Snapshot-style off-chain signed messages pinned to IPFS, CID anchored on-chain).
- The user's existing RAGE "L0–L5 integrity tier" model (Draft → Hashed → CID → Signed → Validated → Promoted) is already an IPFS-native content-addressing pipeline; this guide formalizes it and maps the x402 payment layer (Algorand via GoPlausible's `@x402-avm` packages — note the in-repo module is **pay2play**, not "parsec") onto payment-gated CID delivery.

Key Findings

IPFS reference tooling (current, June 2026)

- **Kubo** (Go; formerly go-ipfs) is the reference implementation and the right choice for a self-hosted node/daemon. It provides the Amino DHT node, HTTP Gateway (trusted + trustless), `/api/v0` RPC, delegated routing, and the `ipfs` CLI. Per the Kubo README (github.com/ipfs/kubo): "we recommend at least 6 GB of RAM and 2 CPU cores… Larger pinsets require additional memory, with an estimated ~1 GiB of RAM per 20 million items for reproviding to the Amino DHT."
- **Helia** (TypeScript) replaced the now-deprecated js-ipfs; use it for in-process/browser nodes. js-ipfs is end-of-life and receives only emergency security fixes. IPFS Blog & News
- **IPFS Cluster** (`ipfs-cluster-service`, `ipfs-cluster-ctl`, `ipfs-cluster-follow`) is the pinset orchestration sidecar — it maintains a global replicated pinset across multiple Kubo daemons. It powers nft.storage and web3.storage/Storacha. Use CRDT consensus mode for production. GitHub
- **Boxo** is the Go SDK of reusable IPFS building blocks (replaces embedding Kubo's internal CoreAPI). **Rainbow** is the specialized high-performance gateway. **IPLD** supplies the data model (DAG-CBOR recommended over DAG-JSON, UnixFS for files). **CAR** (Content

Addressable aRchive) files package content-addressed blocks for bulk/atomic transfer; `ipfs dag export/import` and the `ipfs-car` tool (Storacha) pack/unpack them. IPFS

- **Pinning Service API** is a vendor-agnostic OpenAPI spec (v1.0.0, shipped in go-ipfs 0.8.0+) with `POST /pins`, `GET /pins`, `GET /pins/{requestid}`, `POST /pins/{requestid}`, `DELETE /pins/{requestid}`. Kubo speaks it via `ipfs pin remote`. Pinata, Filebase, and Storacha implement it. IPFS
- **CIDs**: CIDv0 (`Qm…`, base58, dag-pb only) vs CIDv1 (`bafy…`/`bafk…`, self-describing multibase/multicodec/multihash). Best practice for any new project is **CIDv1 base32** (case-insensitive, subdomain-gateway safe); base36 for IPNS/libp2p-key names.
- **IPNS + DNSLink** provide mutable pointers to immutable content. IPNS is a signed, republishable record (`k51…`); DNSLink is a `_dnslink.<domain>` TXT record (`dnslink=/ipfs/<cid>` or `/ipns/<key>`).

ERC-8004 (live on Ethereum mainnet, Jan 29 2026)

- Three on-chain singleton registries per chain: **Identity** (ERC-721 + ERC721URIStorage), **Reputation**, **Validation**. The standard was first proposed August 13, 2025 and went live on mainnet **January 29, 2026**. Per the official EIP-8004 spec (eips.ethereum.org/EIPS/eip-8004) it is authored by **Marco De Rossi (@MarcoMetaMask), Davide Crapis (@dcrapis, davide@ethereum.org), Jordan Ellis (jordanellis@google.com), and Erik Reppel (erik.reppel@coinbase.com)** — i.e., MetaMask, the Ethereum Foundation, Google, and Coinbase.
- Each agent is an ERC-721 token whose `agentURI` (= `tokenURI`) **MUST resolve to a JSON registration file ("agent card")**. The spec explicitly states the URI MAY be `ipfs://<cid>`, `https://…`, or a base64 `data:` URI, and that **content addressing (IPFS) is recommended**; for IPFS URIs the on-chain integrity hash is not required because the CID is itself the hash. QuickNode + 2
- Registration file schema includes `type`, `name`, `description`, `image`, and an `endpoints` array (A2A, MCP, OASF — OASF can itself be `ipfs://{cid}` — and ENS). Reputation `giveFeedback()` emits a `feedbackURI` + optional `feedbackHash`; IPFS feedback URIs omit the hash. Eipsinsight + 2

x402 + Algorand (the payment layer)

- x402 (Coinbase, May 2025; x402 Foundation with Cloudflare, Sept 2025) revives HTTP 402: server returns `402` + `PAYMENT-REQUIRED`, client retries with `PAYMENT-SIGNATURE`, a facilitator verifies/settles on-chain (usually USDC). Per eco.com, "In December 2025, x402 V2 launched with major upgrades, making the protocol multi-chain by default and compatible with legacy payment rails like ACH and card networks." FameEX GitHub
- **Algorand x402 reached full mainnet operational status on February 23, 2026**, when the Algorand Foundation (@AlgoFoundation) announced: "x402 is now fully supported on

Algorand. Spec merged with @coinbase. Facilitator live. Bazaar running. Tooling ready." Algorand was added to the official Coinbase x402 ecosystem page with **GoPlausible as the facilitator**.

- **GoPlausible** publishes the `@x402-avm/*` npm packages (`core`, `avm`, `express`, `hono`, `next`, `paywall`, `extensions`). Per GoPlausible's official docs (github.com/GoPlausible/.github), the AVM implementation "Handles both ALGO and Algorand Standard Assets (ASAs)," uses CAIP-2 network IDs (`ALGORAND_MAINNET_CAIP2`), and integrates wallets "via @txnlab/use-wallet" (Pera/Defly/Lute) with fee abstraction "through atomic transaction groups."
- **Important correction for the user's stack:** there is no discoverable "parsec/parsec-wallet" Algorand x402 project. The AgenticPlace GitHub org's actual payment module family is **`pay2play`** — `pay2play-algo` ("per-request ALGO metering on AVM … x402-shaped HTTP"), `pay2play-glmr` (Moonbeam/GLMR USDC.wh), and `pay2play` (Circle Arc USDC). This guide treats "parsec" as a working alias for that pay2play x402 layer; the user has separately built the first PHP implementation of x402 v1. `github`

Chains in scope for PYTHAI

The `allchain.html` artifact could not be directly retrieved, but the AgenticPlace repos confirm the live chains: **Algorand (AVM)**, **Moonbeam (GLMR, EVM)**, and **Circle Arc (USDC)**, alongside the Ethereum/EVM mainnet target for ERC-8004 registries. RAGE's canonical receipt payload is explicitly **chain-agnostic** (`chain` field), so CIDs can be referenced from whichever chain a given surface settles on.

Details

1. Architecture decision guide

Option	What it is	When to choose	cypherpunk2048 fit
Pinning service (Pinata/Filebase/Storacha)	Remote node holds your pins via Pinning Service API	Fastest start, no ops; fallback/redundancy tier	Sovereignty compromise — documented as *fallback only*
Single self-hosted Kubo	One sovereign node + gateway	Dev, low-volume single surface	Good, but no replication/HA

Option	What it is	When to choose	cypherpunk2048 fit
Self-hosted Kubo + IPFS Cluster (recommended)	N Kubo daemons, each with a cluster sidecar, CRDT pinset, replication factor ⩾ 2–3	Production PYTHAI core: sovereign, HA, no third party	**Best fit** — sovereign infra, mainnet-only, Podman

Recommendation: run a 3-peer Kubo + IPFS Cluster (CRDT) core on your own VPS/edge, replication factor min 2 / max 3, and add a pinning service (Pinata or Filebase) as a documented secondary remote pin for disaster recovery and public-gateway reach. Layer Filecoin (or Storacha, which bundles Filecoin deals) under it for cryptographically-proven durability of high-value artifacts.

2. Step-by-step self-hosted setup (Podman, rootless, Quadlet/systemd)

Podman ⩾ 4.6 with cgroup v2 is required for Quadlet. Run rootless. Create `~/.config/containers/systemd/kubo.container`:

ini

```
[Unit]
Description=kubo_ipfs_node
After=local-fs.target

[Container]
Image=docker.io/ipfs/kubo:latest
ContainerName=kubo
AutoUpdate=registry
Volume=%h/.ipfs:/data/ipfs:Z
Volume=%h/ipfs_export:/export:Z
PublishPort=127.0.0.1:5001:5001
PublishPort=127.0.0.1:8080:8080
PublishPort=4001:4001
PublishPort=4001:4001/udp
UserNS=keep-id
Environment=IPFS_PROFILE=server

[Service]
Restart=always
TimeoutStartSec=900

[Install]
WantedBy=default.target
```

Then `systemctl --user daemon-reload && systemctl --user enable --now kubo.service`. Keep the RPC API (`5001`) bound to loopback only. Set `loginctl enable-linger $USER` so rootless units survive logout. (This Quadlet pattern is the one proposed for Kubo in ipfs/kubo issue #10561.)

Initialize CIDv1 as the default and set the gateway to trustless/verifiable mode where the node is public:

bash

```
podman exec kubo ipfs config --json Import.CidVersion 1
podman exec kubo ipfs config Gateway.NoDNSLink false
# public node hardening: serve only verifiable raw/CAR (no deserialized HTML)
```

IPFS Cluster sidecar — `~/.config/containers/systemd/cluster.container` running `docker.io/ipfs/ipfs-cluster:latest`, connected to the Kubo RPC, sharing a `CLUSTER_SECRET` (32-byte hex: `od -vN 32 -An -tx1 /dev/urandom | tr -d ' \n'`) across all peers, `--consensus crdt`, with `trusted_peers` set to your own peer IDs (never `*`). Pin via `ipfs-`

`cluster-ctl pin add --replication-min 2 --replication-max 3 <cid>`; check with `ipfs-cluster-ctl status <cid>`. IPFS Cluster IPFS Cluster

Peering: add your cluster peers to each Kubo's `Peering.Peers` for sticky connections. **IPNS keys:** `ipfs key gen --type=ed25519 surface-bankon`; publish with `ipfs name publish --key=surface-bankon /ipfs/<cid>`. **DNSLink:** add TXT `_dnslink.bankon.pythai.net "dnslink=/ipns/k51…"` (low TTL ~60 s), so `bankon.pythai.net` resolves to current content while CIDs stay immutable. DEV Community

3. Programmatic pinning / upload patterns (Python ≥ 3.12)

JSON metadata, NFT/token metadata, documents, and AI artifacts should be added with CIDv1 and pinned to the cluster:

```python
# Python 3.12+, snake_case, talks to local Kubo RPC (sovereign, no third party)
import json, httpx

KUBO_RPC = "http://127.0.0.1:5001/api/v0"

def add_json(obj: dict) -> str:
    canonical = json.dumps(obj, sort_keys=True, separators=(",", ":")).encode()
    resp = httpx.post(
        f"{KUBO_RPC}/add",
        params={"cid-version": 1, "pin": "true", "raw-leaves": "true"},
        files={"file": ("metadata.json", canonical, "application/json")},
    )
    return resp.json()["Hash"]  # bafy… CIDv1
```

For **bulk/atomic** uploads (e.g., a whole agent bundle or a RAGE "manifest" of capsules), pack a CAR and import it so the whole DAG shares one root CID and is added atomically:

```bash
npx ipfs-car pack ./agent_bundle --output bundle.car   # deterministic root CID
podman exec kubo ipfs dag import /export/bundle.car
ipfs-cluster-ctl add --car bundle.car                  # cluster supports CAR import
```

Canonicalize JSON before hashing/signing (sorted keys, UTF-8) so the CID is reproducible — this matches RAGE's stated best practice. github

4. On-chain content-addressing best practices

- Store the **CIDv1** in the contract and expose it as `ipfs://<cid>` from `tokenURI`/`agentURI`.

For collections, an `ipfs://<dirCID>/<id>` base-URI directory pattern works; for per-token, use OpenZeppelin `ERC721URIStorage._setTokenURI`. Speedrunethereum

- **Immutability:** because cypherpunk2048 forbids admin keys and upgradeable proxies, prefer **immutable per-token `ipfs://` URIs set at mint** (a `setBaseURI`-style owner function contradicts the no-admin-keys rule). If content must evolve, point the on-chain URI at an **IPNS** name (`ipns://k51…`) once and update the IPNS record off-chain — mutability lives in IPNS, not in an admin-controlled contract function. DEV Community
- Test all Solidity with **Foundry** (`forge test`), deploy to **mainnet only**.

5. Durability / permanence layer — IPFS+Filecoin vs Arweave

- **IPFS alone has no persistence guarantee** — data survives only while pinned. Your Cluster provides that within your infra. Medium Medium
- **Filecoin** adds economic storage deals with Proof-of-Replication / Proof-of-Spacetime; "Filecoin Pin" gives daily Proof-of-Data-Possession (PDP) cryptographic proofs and is IPFS/CID-compatible — the Filecoin docs even ship an ERC-8004 agent-card registration cookbook. Deals are time-bounded and must be renewed/funded. Open Source For You + 2
- **Arweave** is pay-once, store-forever (blockweave + endowment), default-public and immutable; best for legal/governance archives and anything needing 100+ year availability. The user already runs Arweave/AR.IO. Aicerts + 2
- **Decision rule:** use **IPFS+Filecoin** for the working, content-addressed, possibly-evolving layer (agent cards, model outputs, listings, day-to-day governance) where CID portability and cheap retrieval matter; use **Arweave/AR.IO** for write-once permanence of ratified DAIO governance records, identity attestations, and anything that must never be re-pinned or renewed. Many teams store the high-value original on Arweave and the working copy + previews on IPFS, recording both pointers. Freeport Studio 721

6. DNSLink + IPNS for mutable pointers

Set each surface's human-readable subdomain via DNSLink to an IPNS key you control; publish new CIDs to that key as content evolves, never touching DNS again. This keeps content-addressed immutability (each CID is fixed) while giving a stable, friendly entry point. DEV Community

Per-surface mapping

AgenticPlace (agenticplace.pythai.net) — ERC-8004 agent marketplace

Content	Format	CID handling	On-chain reference / chain
Agent registration file / "agent card"	JSON (ERC-8004 `registration-v1`)	CIDv1, pinned to cluster,	`agentURI`/`tokenURI` = `ipfs://<cid>` in ERC-8004 Identity

Content	Format	CID handling	On-chain reference / chain
		optionally Filecoin-pinned	Registry (ERC-721) on Ethereum/EVM mainnet
Agent artifacts (weights refs, prompts, outputs)	files / CAR bundle	CAR-packed, one root CID	Referenced from agent card `endpoints`/OASF `ipfs://{cid}`
Marketplace listing metadata	JSON	CIDv1	Listing contract stores CID; `ipfs://`
NFT-style media	image/media	CIDv1 dir	`image` field `ipfs://<cid>`
Reputation feedback detail	JSON	CIDv1	`feedbackURI` in Reputation Registry (hash omitted for IPFS)

Use immutable `ipfs://` for ratified cards; use `ipns://` if a card must be updatable without an admin-keyed contract.

mindX (mindx.pythai.net) — autonomous AI cognitive system with API

Content	Format	CID handling	On-chain reference / chain
AI-generated outputs/artifacts from API	JSON/files	Canonicalize → CIDv1 → cluster pin	CID embedded in a RAGE-style signed receipt
Model output provenance	"Capsule" + "Receipt"	CID (L2) → wallet signature over CID (L3)	Receipt's `ipfs_cid` + signature, chain-agnostic
Verifiable artifact bundles	CAR manifest	Merkle/bundle root CID (L5 "Promoted")	Optional anchor/mint via SubMinter

mindX's API should, on each generation, (1) canonicalize the output, (2) `add` with CIDv1 + pin to cluster, (3) emit a signed receipt binding wallet ↔ action ↔ CID, matching RAGE's L0–L5 integrity tiers (default-serve L3+). Payment-gate the API with x402 so a caller pays (USDC on Algorand/Base) and receives the CID + retrieval URL on `200` — x402 and CID delivery compose cleanly because the paid resource is simply the content-addressed artifact. github

BANKON (bankon.pythai.net) — identity & payment layer + DAIO governance

Content	Format	CID handling	On-chain reference / chain
Identity documents	JSON (+ client-side	CIDv1, cluster pin; Arweave	CID in identity contract;

Content	Format	CID handling	On-chain reference / chain
/ metadata	encryption for PII)	for permanent attestations	`ipfs://`
DAIO governance proposals	JSON	CIDv1, pinned	Proposal CID anchored on-chain / in Snapshot space
DAIO votes	EIP-712 signed messages	Pinned to IPFS (Snapshot pattern)	Signatures on IPFS; result CID anchored
Governance config/space	JSON	CIDv1	ENS/DNS text record → CID (Snapshot space pattern)
x402 payment-gated content	any	CID	Delivered on `200` after settlement

DAIO governance deployment guidance: follow the proven Snapshot model — proposals and votes are cryptographically-signed (EIP-712) messages **stored on IPFS, not on-chain**, giving gasless, tamper-evident, fully-auditable governance; only binding execution (treasury, parameter changes) touches the chain (Aragon/Tally/SafeSnap pattern). For BANKON's DAIO: pin proposals/votes to your sovereign cluster (with a pinning-service mirror so records survive if your nodes are offline — Snapshot's known centralization risk is its IPFS hub, which your self-hosted cluster + Filecoin mitigates), anchor each proposal's root CID on the mainnet DAIO contract, and archive ratified outcomes to Arweave for permanence. Encrypt any PII client-side before pinning, since IPFS/Arweave content is public by default. DEXTools + 3

x402 / payment intersection: BANKON's payment layer and mindX/AgenticPlace can monetize content-addressed delivery with x402 — a `402` returns payment requirements, settlement happens on Algorand (GoPlausible `@x402-avm`, USDC ASA via atomic transaction groups) / Moonbeam (USDC.wh) / Arc (USDC) via the `pay2play` module, and the `200` response hands back the `ipfs://<cid>`. The CID is the deliverable; x402 is the turnstile.

Recommendations

1. **Stand up the 3-peer Kubo + IPFS Cluster core first** (rootless Podman Quadlets, CRDT, replication 2–3, CIDv1 default, loopback-only RPC, trustless public gateway). Benchmark: all three peers show `PINNED` for a test CID and the public gateway serves `?format=car` verifiably.
2. **Add a pinning-service mirror (Pinata or Filebase) via `ipfs pin remote`** as documented fallback, and wire **Filecoin Pin** for high-value artifacts (agent cards, ratified governance). Threshold to escalate to Arweave: any record that is legally/constitutionally permanent or must never be renewed.

3. **AgenticPlace:** mint ERC-8004 agents with `agentURI = ipfs://<cid>`; CAR-bundle agent artifacts; keep cards immutable unless updatability is required, then use `ipns://`. Test registry interactions with Foundry, deploy registries/marketplace to mainnet.
4. **mindX:** make CID generation + signed receipt a non-optional step in the API response path; default retrieval to integrity tier L3+; x402-gate the endpoint.
5. **BANKON DAIO:** deploy Snapshot-style off-chain signed governance pinned to your cluster + mirror, anchor proposal CIDs on the mainnet DAIO contract, archive ratified results to Arweave, encrypt PII before pinning.
6. **Naming/standards:** confirm whether "parsec/parsec-wallet" is an internal alias for the `pay2play` x402 module; align repo naming (flat snake_case), Apache-2.0 license, no admin keys / no proxies, mainnet targets, Python ⩾ 3.12, Foundry, Podman.

Caveats

- `allchain.html` **could not be directly retrieved**; the in-scope chains (Algorand, Moonbeam/GLMR, Circle Arc, plus Ethereum/EVM for ERC-8004) are inferred from the AgenticPlace `pay2play` repositories, not from that artifact. Verify exact chain IDs/contract addresses against the page itself.
- **No public "parsec/parsec-wallet" Algorand x402 project exists**; the discoverable module is `pay2play`. Treat "parsec" as an internal name pending confirmation.
- ERC-8004's Validation Registry is still under active revision (TEE-community discussion); design validation flows to be swappable. GitHub
- IPFS/Arweave content is **public by default** — never pin unencrypted PII or secrets.
- Filecoin "perpetual storage" is still contract/funding-based, not literally forever; Arweave is the true pay-once permanence option.
- Some ecosystem adoption/volume figures are from secondary/analyst sources; the authoritative dates used above are the Algorand Foundation's February 23, 2026 full-support announcement, the x402 V2 launch in December 2025 (per eco.com), and ERC-8004's January 29, 2026 mainnet go-live.